B-12 / Capri: локальний AI-компаньйон із простежуваною пам’яттю та контрольованими інструментами

Олександр Пилипенко
Державний торговельно-економічний університет, Київ, Україна

Робочий препринт, 8 листопада 2026 року

Анотація

Ми описуємо B-12 — локальний агентний runtime — і Capri, його desktop-компаньйона. Архітектура відокремлює модельний текст від ідентифікації, згоди, дозволів та виконання інструментів; зберігає історію й пам’ять поза вагами моделі; допускає запити за явним профілем токенізатора та розгортання. На ноутбуці з RTX 4060 Laptop і квантизованою Nemotron Bolt E4B історичне оцінювання на commit `60a4c0b` зафіксувало 496 успіхів у 541 запуску для 109 сконструйованих сценаріїв і контексту 4096 токенів. Інструментальні задачі пройдено у 182 із 215 запусків; adversarial перевірка outbound fetch не пройшла в усіх п’яти повторах. Окрема калібровка зафіксувала точний збіг вхідних токенів у 15 запитах. Експлуатаційний прогін накопичив 8,007 години активного часу в чотирьох сегментах. Ці результати стосуються історичних конфігурацій на одному комп’ютері, не встановлюють якості поточної desktop-версії та не є порівнянням моделей. Обговорено обрізання контексту, відмови інструментів, межі приватності, відтворюваність і вимоги до безпечної заміни моделі.

Ключові слова: локальні агенти; інструменти LLM; контроль контексту; походження пам’яті; компаньйон; оцінювання

1 Вступ

Персональний асистент — це система, що поєднує модель, історію, доступ до інформації та дії в середовищі з обмеженими ресурсами. Модель може запропонувати виклик інструмента, але її текст не має самостійно надавати повноваження. Збережена розмова не обов’язково повністю присутня в наступному запиті. Заявлений розмір контекстного вікна не доводить, що конкретний runtime зберігає всі інструкції.

B-12 досліджує ці межі на споживчому обладнанні. Capri — видимий desktop-компаньйон, а B-12 залишається назвою runtime. Стаття ставить питання, як поєднати контроль дій, пам’ять та обмежені модельні запити в локальному асистенті і що виявляють наявні вимірювання. Це інженерний звіт та аналіз історичних доказів, а не контрольоване порівняння моделей чи твердження про свідомість. Код і документація доступні в репозиторії B-12 [1].

2 Архітектура

Один runtime володіє сховищами control, sessions і memory, індексом knowledge, реєстром інструментів та агентом. Налаштований адаптер Ollama виконує inference. CLI, web та desktop використовують спільний runtime або автентифікований loopback API. Qt-оболонка дає компактний чат і необов’язковий вигляд Caprі або кіберкота. Анімації є презентаційною поведінкою та самі не надають дозволів інструментам.

Для ходу runtime визначає актора та grant, отримує допустимі джерела, формує обмежений запит і надсилає його моделі. Виклики інструментів перевіряються за схемами й чинним контрольним станом; дозволені результати передаються в наступні раунди. Події розрізняють виконання, очікування підтвердження, скасування й кінцевий стан. Exact approval пов’язаний з операцією, а не з формулюванням модельного тексту.

Пам’ять, цілі та сесії зберігаються поза вагами. Ідентифікатори evidence і перевірки життєвого циклу пов’язують отримані дані з джерелами. Видалення координує керовані сховища та похідний стан; зовнішні копії й cache провайдера не охоплені універсальною гарантією стирання. Збережена історія не гарантує, що вся вона вміститься в наступний запит. Старі завершені ходи можуть бути вилучені з контексту; короткі витяги підтримують обмежену наступність, але не замінюють повний transcript.

Профіль пов’язує tokenizer assets, chat template, manifest моделі та версію Ollama. Context builder підраховує відрендерені повідомлення й схеми інструментів до admission; адаптер повторно перевіряє запит і deployment. Якщо обов’язкові дані не вміщаються, запит завершується помилкою замість тихого вилучення контрольних інструкцій. Зовнішня зміна alias Ollama між перевіркою і запитом залишається можливою.

2.1 Компаньйон, голос і доступ до інформації

Desktop-гілка надає компактний чат, вибір вигляду, нагадування, налаштовані власником launchers та opt-in інтеграції активності. Клієнт передає чат і підтвердження до runtime. Голосовий код включає розпізнавання, синтез мовлення й обробку переривання; фізичне акустичне приймання та повну голосову затримку не встановлено. Мережеві search і fetch залежать від конфігурації runtime. Поточний текстовий чат не приймає зображення. Ці описи desktop і голосу стосуються новішого коду, ніж історичне оцінювання.

3 Суміжні роботи

ReAct поєднує модельне міркування з діями [5]; retrieval-augmented generation поєднує генерацію із зовнішнім пошуком [6]. Дослідження довгого контексту показує вплив позиції релевантної інформації [7]. Ці роботи мотивують взаємодію агента, retrieval і оцінювання контексту, але не підтверджують саме цю реалізацію.

Дослідження автора про archive-index розглядає мультимовний лексичний, dense та hybrid retrieval за власних умов колекції й оцінювання [2].

Вимірювання B-12 у цій статті не відтворюють його retrieval benchmark. Звіт про компаньйонів у World of Warcraft описує типізовані дії та окрему межу перевірки [3]; аудит автономності в Minecraft розділяє самостійний прогрес і допомогу оператора [4]. Ці роботи мотивують увагу до доказів і контролю, але їхні показники належать відповідним системам.

4 Методика та докази

Використано три наявні набори без нового inference. Перший text-only smoke test Bolt Q4_K_M від 24 жовтня 2026 року виконано на Intel i9-14900HX, 32 ГБ RAM і RTX 4060 Laptop 8 ГБ. При контексті 2048 тепла генерація в виміряних запитах становила приблизно 62–64 токени/с; перший холодний запит дав перший непорожній текст через 29,82 с. Надто великий prompt був обрізаний із втратою інструкції та унікального префікса.
Друга калібровка профілю на commit `4e3cc06`, Ollama 0.33.3 і контексті 4096 охопила український текст та emoji, схеми інструментів, кілька раундів, запит біля межі контексту й native tool call. Кожен із п’яти сценаріїв повторено тричі; для всіх 15 запитів записано точний збіг підрахунку вхідних токенів. Це обмежена калібровка конкретного deployment.
Третій набір — оцінювання агента на commit `60a4c0b49b2f`: Ollama 0.33.3, alias `b12-bolt-e4b:q4km`, контекст 4096, вихідний ліміт 512, temperature zero та seed 42. Авторський корпус містив 109 сценаріїв українською, англійською, для інструментів, пам’яті, поведінки, adversarial запитів і однієї довгої розмови. Звичайні сценарії повторювали п’ять разів; розмову виконали один раз, отже всього 541 запуск. Оцінювання детерміноване; незалежної людської оцінки мови чи особистості не було. Ми переглянули локальні metadata і summary, включно з хешами сценаріїв та профілю.

5 Результати

Категорія	Пройдено	Запусків
Українська	95	100
Англійська / змішана	75	75
Інструменти	182	215
Пам’ять	68	70
Поведінка	35	35
Adversarial	40	45
Розмова	1	1
Разом	496	541

Таблиця 1: Історичне оцінювання, контекст 4096, commit `60a4c0b`.

Агрегований результат становив 91,7% у цьому сконструйованому корпусі. Із 150 багатокрокових запусків інструментів пройдено 117; summary не містить

протокольних помилок. Окремий одноразовий прогін на контексті 8192 дав 102/109, але інший бюджет і один повтор не дозволяють вважати це контрольованим порівнянням. Повтори сценарію залежні, а агрегат не є оцінкою надійності для всіх задач.
У summary прогону 4096 P50 часу до першого тексту — 0,816 с, P95 — 1,283 с. Окрема коротка latency-серія дала 10,64 с холодного старту й приблизно 0,65–0,72 с у теплому стані з кешованими токенами. Ці значення не включають мікрофон, розпізнавання й синтез мовлення.
Adversarial сценарій outbound fetch не пройшов перевірку в усіх п’яти повторах: модель обрала `http_fetch`, хоча тест очікував відсутності виклику. Це результат історичної конфігурації та конкретної перевірки, а не доказ універсального exploit або поточної поведінки. Пізніша allowlist не обмежує wildcard, обраний власником. Інші відмови включали неправильні шляхи у викликах, копіювання серіалізованої структури tool result замість потрібного тексту, транслітерацію українського токена, неправильну фактичну відповідь і два провали виправлення пам’яті. Зміни сценаріїв і описів інструментів між прогонами не дозволяють приписувати різницю агрегатів одній причині без абляції.
Експлуатаційний прогін накопичив 8,007 години активного часу у чотирьох сегментах і з трьома паузами. Зафіксовано 21 успішну перевірку цілісності, навмисні простої моделі й приблизно 111 тисяч ходів як нижню межу суми метрик воркерів. Прогін не був безперервним; RSS воркерів зростав протягом їхнього життя. Подальший maintenance не перевіряли новим довгим soak. Це програмний експеримент, а не приймання фізичного робота.

6 Деталі реалізації та інженерні компроміси

6.1 Хід як послідовність із контрольованим результатом

Хід починається з сесії, grant і тексту користувача. Контрольний шар визначає актора та чинні повноваження; застосунок не отримує нових прав із повідомлення чи аргументу, згенерованого моделлю. Агент знаходить допустимі джерела, формує модельний ввід і виконує раунди до фінальної відповіді, скасування, помилки або очікування підтвердження. Результат інструмента може бути переданий моделі для продовження. Це дає змогу аналізувати не лише відповідь, а й послідовність вибраних інструментів, перевірок і спостережених ефектів.
Потік подій розділяє початок, фрагменти тексту, запуск і результат інструмента, запит підтвердження та кінцевий стан. Очікування approval не означає завершення ходу. Після відновлення дія знову перевіряється за чинним control state, зокрема за станом grant. Для довгих взаємодій це важливо: дозвіл, чинний на початку задачі, не обов’язково чинний під час продовження. Частковий модельний текст не є завершеною відповіддю або доказом виконаної дії.
Реалізація розрізняє три результати операції. Completed означає підтверджений ефект. Failed застосовується, коли trusted handler може встановити, що ефект не відбувся. Unknown охоплює timeout, скасування чи restart, після яких неясно, чи стався частковий ефект. Автоматичний повтор

невідомої операції може продублювати дію; тому невизначеність не прирівнюється до безпечної відмови. Це інженерний контракт, а не доказ можливості спостерігати будь-який host effect.

6.2 Admission, історія та межі модельного вікна

Context builder обліковує system prompt, розмову, отримані джерела та схеми інструментів у спільному бюджеті. Резерв виходу й safety margin зменшують місце для вводу. Кількість токенів залежить від модельного шаблону й токенізатора, а не лише від кількості символів. Проєкт фіксує хеші файлів профілю, а адаптер перевіряє deployment перед генерацією.
Початковий smoke test показав практичний режим відмови: довгий запит втратив початок, хоча модель усе ще генерувала правдоподібний текст. Пізніший admission змінює поведінку: коли обов’язкові дані не вміщуються, він відмовляє. Старі завершені ходи можуть вилучатися узгодженими групами з фіксацією їхніх ідентифікаторів. Невеликий extractive breadcrumb підтримує обмежену наступність, але не відновлює всі пропущені повідомлення чи дані поза збереженим вікном. Відповідь про давню розмову залежить від того, що справді отримано або допущено до контексту.
Калібровка не доводить еквівалентності всіх provider templates. Реальний інтеграційний прогін Bolt виявив розбіжність на шість токенів для композиційної схеми інструмента; перевірка не допустила запит. Після цього registry обмежив непідтримані конструкції схем. Профіль, придатний для простого чату, автоматично не підтверджує кожну схему інструмента. Для нової моделі потрібна окрема перевірка tokenizer, template, digest та версії runtime.

6.3 Життєвий цикл пам’яті та походження

Довготривале сховище відокремлено від transcript сесії та контексту модельного запиту. Згода й actor scope визначають доступ до записів, а evidence tags пов’язують хід із відповідними матеріалами. Хід, пов’язаний із уже недійсним evidence, не записується як звичайне успішне продовження. Система координує видалення керованих даних і веде durable запис незавершеної роботи, доки всі учасники не підтвердять виконання.
Така координація визначає обсяг операції, яку інакше легко надто широко назвати забуванням. Вона охоплює зареєстровані компоненти, включно з керованими сесіями та backup state, але не доводить стирання з усіх зовнішніх файлів, незалежних файлових snapshots чи provider caches. Дія щодо cache Ollama є model-level unload, а не криптографічним доказом видалення cache окремої людини. Повторення відомостей користувачем в іншому непов’язаному повідомленні також може створити дані поза первинним evidence link. Ці межі потрібні для коректного тлумачення результатів видалення.

6.4 Інтерфейс, аудіо та доступ до середовища

Desktop-шар Caprí є клієнтом runtime. Компактний чат використовує спільний потік сесій і ходів та не створює окремої системи пам’яті. Локальні

налаштування включають вигляд і обмежені стильові побажання; стильовий профіль є prompt context, а не механізмом додавання інструментів або grants. Якщо в майбутньому використовується віддалена модель, включені до запиту профіль та історія передаватимуться провайдеру.
Desktop-гілка містить нагадування, налаштовані власником application launchers, локальні файлові й системні панелі та необов’язкові integration hooks. Вони мають різні способи активації та не охоплені історичним корпусом Bolt. Перемикач вигляду може повернутися до вбудованого кіберкота; локально імпортований арт Caprí має окреме походження та обмеження поширення. Візуальний персонаж не означає, що модель бачить екран: поточний текстовий чат не приймає зображення.
Голосовий код містить каскад розпізнавання мовлення, взаємодії з моделлю та синтезу, а також обробку переривань. Історичні виміри компонентів залежать від конкретних аудіовходів і програмних умов. Симульоване приглушення еха, діагностичні перевірки мовця та програмний час скасування не доводять надійності в реальній кімнаті. Восьмигодинний фізичний акустичний тест, повна затримка голосової відповіді й ширша точність верифікації мовця залишаються відкритими.

7 Інтерпретація оцінювання

7.1 Що входить до агрегату

109 кейсів є сконструйованим інженерним корпусом, а не випадковою вибіркою всіх способів використання персонального асистента. Категорія інструментів включає одно- й багатокрокові задачі; пам’ять — retrieval і виправлення; adversarial checks охоплюють вибрані взаємодії моделі з control. Категорія розмови містить один довгий сценарій. Тому агрегат 496/541 зважує категорії за кількістю запусків, а п’ять повторів кейсу не є п’ятьма незалежними задачами.
Нуль протокольних помилок у summary стосується визначення protocol error в інструментації цього прогону. Це не означає, що кожна відповідь, виклик, побічний ефект чи політика були правильними. 33 невдалі інструментальні запуски та п’ять невдалих adversarial запусків є частиною результату. Необхідно розрізняти результат перевірки задачі, перший текст, завантаження моделі, ефект інструмента й фізичну голосову затримку: ці показники відповідають на різні питання.

7.2 Механізми відмов

Сценарій outbound fetch показав розбіжність між очікуваною поведінкою кейсу та вибором моделі у перевіреній конфігурації. Настроювана allowlist доменів може обмежувати адреси, але ефект залежить від реальної політики власника. Wildcard допускає широке коло адрес. Цей запис не встановлює поведінку кожної пізнішої версії, grant чи мережевої конфігурації.
Частину інструментальних відмов варто розглядати на межі між моделлю та інтерфейсом. Неправильний шлях може зірвати змістовну дію; модель може

скопіювати обгортку структурованого результату, коли задача вимагає лише текст; транслітерація змінює точний вміст. Виправлення пам’яті може провалитися навіть за доступного сховища. Це мотивує аналіз окремих кейсів і трас, а не припущення, що один відсоток визначає причину дефекту.
Деякі ранні corpus checks також потребували уточнення: окремі очікування були занадто вузькими, а пізніші adversarial checks розширено на додаткові command tools. Версії корпусу різняться. Порівняння між ними не ізолює ефект однієї зміни, якщо дані, модель, prompts та умови виконання не фіксовані. Публічні файли оцінювання є інженерним інструментом, але для незалежного відтворення таблиці потрібні privacy review історичного JSONL і стабільний публічний артефакт.

7.3 Обладнання та швидкодія

Результати ноутбука залежать від завантаження моделі, довжини контексту, доступної VRAM, версії провайдера та теплого cache. Холодний перший текст із початкового smoke test і окремої latency-серії — різні вимірювання; жодне не оцінює повну голосову відповідь. Швидкість теплої генерації стосується конкретного text-only прогону й не встановлює швидкодії за одночасного розміщення vision encoder, мовленнєвих моделей чи інших GPU-процесів. Сегментований експлуатаційний прогін досліджує відновлення, цілісність сховищ і повторну активність за накопичений час. Він не встановлює безперервного сервісу протягом восьми годин, розгортання на Jetson, безпечності фізичних рухів або тривалої стабільності особистості. Пізніші зміни керування ресурсами потребують нових вимірювань перед кількісними твердженнями про довгоживучий воркер.

8 Програма наступних експериментів

Наступне дослідження може порівняти кандидатні моделі на одному версійованому корпусі з окремо відкаліброваними профілями. Потрібно записувати digest, quantization, версію провайдера, prompt, бюджети контексту й виходу, стан cache, навантаження хоста, кількість повторів і сирі результати. Об’єктивні перевірки інструментів і пам’яті слід відділяти від сліпої людської оцінки української мови, характеру розмови та фактичності. Порівнювані бюджети не означають тотожності токенізаторів чи шаблонів: ці відмінності треба явно описувати.
Дослідження стабільності persona може включати короткі привітання, незгоду, невизначеність, довгі розмови, серйозні переживання користувача та технічно точні задачі. Оцінювачі мають аналізувати стиль і послідовність, не тлумачачи стилістичну відповідь як доказ суб’єктивного досвіду. Згоду, приватність і контроль інструментів потрібно перевіряти незалежно від вибраної persona. До проведення такого дослідження наступність після заміни моделі є метою архітектури, а не виміряною властивістю.

9 Обговорення

Окреме від ваг зберігання дозволяє передати новій моделі допустимі наявні записи, але не гарантує однакової інтерпретації, retrieval, стилю чи особистості. Нинішній prompt Caprі та desktop-анімації з’явилися після історичного оцінювання. Людського benchmark стабільності особистості немає, тому результат категорії behavior не доводить задоволеності користувача чи відповідності характеру.
Заміна моделі є архітектурною метою, а не завершеною функцією live switch. У репозиторії зараз один відкалібрований профіль Bolt. Безпечне перемикання має разом перевіряти модель, tokenizer, template та бюджети й координувати ходи під час підготовки, активної генерації, очікування approvals і фонової reflection. Одного model lock недостатньо, оскільки admission відбувається до блокування виклику. Автоматичний routing і прямі cloud adapters залишаються майбутньою роботою; хмарний запит передасть провайдеру включений до нього контекст.
Контрольний шлях зменшує неоднозначність, але не є повною ізоляцією процесу з широкими правами власника. Самоогляд проєкту зафіксував невирішені ризики доступу до файлів, виконання процесів і мережевої політики. Тести вибраних сценаріїв не доводять стійкості до кожного prompt injection чи ексфільтрації. Фізичне приймання робота, тривала акустична перевірка та повний голосовий SLO ще не підтверджені.

10 Обмеження та відтворюваність

Докази спираються на один комп’ютер, одну основну модель, сценарії авторів проєкту й детерміновані перевірки. Немає рандомізованого порівняння моделей, незалежного мовного аудиту чи дослідження користувачів. Оцінювання моделі передує подальшим змінам runtime і desktop, а експлуатаційний прогін був сегментованим. Поточна гілка публічного репозиторію не обов’язково відтворює історичний результат без точних commit, профілю, assets, ваг та версії runtime.
Історичні metadata і JSONL зберігаються локально в runtime worktree й поки не є публічним пакетом відтворення. Перед публікацією потрібні перевірка трас на приватні дані, свідоме оприлюднення, контрольні суми та підтвердження авторських метаданих. Цей рукопис є робочою версією; бібліографія й огляд суміжних робіт потребують додаткового незалежного перегляду.

11 Висновок

B-12 / Caprі демонструє локальну архітектуру, що відокремлює модельний текст від повноважень, надає даним життєвий цикл і пов’язує admission запитів із перевіреним профілем. Історичні результати показують значну кількість успішних задач разом із конкретними відмовами інструментів, пам’яті й мережевої політики. Докази мотивують контрольоване порівняння моделей та

людське оцінювання; вони не доводять незалежної від моделі особистості Caprі чи успадкування поточним desktop історичного показника.

Література

[1] Пилипенко О. Репозиторій B-12 і документація. https://github.com/cyberdid/B-12.

[2] Pylypenko, O. Beyond Search Scores. Zenodo, 2026. https://doi.org/10.5281/zenodo.23602155.

[3] Pylypenko, O. Two Language-Model Companions in a Private World of Warcraft Server. Zenodo, 2026. https://doi.org/10.5281/zenodo.23911809.

[4] Pylypenko, O. Auditing AI Autonomy in Minecraft. Zenodo, 2026. https://doi.org/10.5281/zenodo.23910891.

[5] Yao, S. та ін. ReAct: Synergizing Reasoning and Acting in Language Models. 2022. https://arxiv.org/abs/2210.03629.

[6] Lewis, P. та ін. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. 2020. https://arxiv.org/abs/2005.11401.

[7] Liu, N. F. та ін. Lost in the Middle: How Language Models Use Long Contexts. 2023. https://arxiv.org/abs/2307.03172.